

ورود به دنیای هوش مصنوعی

دوره‌ی مقدماتی و رایگان برای عموم علاقه‌مندان

جلسه‌ی نهم: پردازش زبان طبیعی، وقتی ماشین حرف ما را می‌فهمد

علی شریفی زارچی

رئیس کمیته‌ی علمی بین‌المللی المپیاد جهانی هوش مصنوعی

عضو کمیته‌ی علمی بین‌المللی المپیاد جهانی کامپیوتر

مرور جلسه‌ی قبل

- تصویر برای کامپیوتر، جدولی از اعداد است
- کانولوشن: فیلترهای کوچک یادگیرنده که روی تصویر می‌لغزند
- هر فیلتر یک نقشه‌ی ویژگی می‌سازد؛ عمق خروجی = تعداد فیلترها
- لایه‌ی تصمیم + Pooling + کانولوشن: CNN
- امروز: از پیکسل به کلمه – ماشین چطور «زبان» را می‌فهمد؟

پردازش زبان، هر روز و همه جا

جست و جو و چت

جست و جوی معنایی
چت بات پشتیبانی، ChatGPT

دستیارها

سیری و دستیارهای صوتی
تبدیل گفتار به متن

ترجمه

مترجم گوگل و دیپال
بیش از ۱۰۰ زبان

جای تصویر

نمای گفت و گو با یک چت بات

جای تصویر

اسکرین شات دستیار صوتی یا موج صوتی

جای تصویر

نمای مترجم گوگل روی یک جمله فارسی

همه‌ی این‌ها یک کار مشترک دارند: تبدیل زبان آدم‌ها به عدد، و بعد پیش‌بینی.

مسیر تاریخی: از قواعد دستی تا مدل‌های زبانی

- دهه‌ی ۱۹۶۰: ELIZA: چت‌باتی با چند قاعده‌ی ساده که وانمود می‌کرد روان‌درمان‌گر است
- دهه‌ی ۱۹۹۰: روش‌های آماری: شمارش کلمات و احتمال‌ها به‌جای قواعد دست‌نویس
- ۲۰۱۳: word2vec: معنا به شکل بردار؛ شاه - مرد + زن $\approx$ ملکه
- ۲۰۱۷: ترنسفورمر: مقاله‌ی «Attention Is All You Need» همه‌چیز را عوض کرد
- ۲۰۱۸ به بعد: BERT و GPT، پیش‌آموزش روی متن عظیم، سپس تطبیق برای هر کار

داده‌های مرجع پردازش زبان

نقش	کار	اندازه	مجموعه داده
تمرین کلاسیک دسته‌بندی متن	تحلیل احساسات	۵۰ هزار نقد	IMDB Reviews
درک مطلب از روی متن	پرسش و پاسخ	۱۰۰ هزار پرسش	SQuAD
مرجع مقایسه‌ی مدل‌ها	سنجش فهم زبان	چند وظیفه	GLUE / SuperGLUE
مسابقه‌ی سالانه‌ی ترجمه‌ی ماشینی	ترجمه	میلیون‌ها جمله	WMT
سخت اصلی مدل‌های زبانی بزرگ	پیش‌آموزش	پتابایت متن وب	Common Crawl
پایه‌ی مدل‌های فارسی مثل ParsBERT	پیش‌آموزش فارسی	۱~ میلیون مقاله	ویکی‌پدیای فارسی

سوال کلیدی

تصویر از اول عدد بود؛

اما «کلمه» را چطور به عدد تبدیل کنیم؟

چرا زبان این قدر سخت است؟

کنایه و استعاره

«دستت درد نکنه!»
گاهی تشکر است،
گاهی گلایه!

وابستگی دوربرد

«کتابی که دیروز از
کتابخانه گرفتم را خواندم»
فعل به کجا برمی‌گردد؟

ابهام

«شیر»: حیوان، نوشیدنی، یا شیر آب؟
از روی بافت (Context) می‌توان فهمید

راه نخست: نمایش One-hot برای کلمات

- نمایش One-hot: هر کلمه یک بردار بلند با یک ۱ و بقیه ۰
- فارسی ده‌ها هزار کلمه دارد ← بردارهای ده‌ها هزار بعدی که تقریباً همه صفرند
- مشکل بدتر: هیچ شباهتی ثبت نمی‌شود
- فاصله‌ی «گربه» تا «سگ» دقیقاً برابر فاصله‌ی «گربه» تا «پنجره» است!
- ما نمایشی می‌خواهیم که «معنا» را در دل خود داشته باشد

راه دوم: کیسه‌ی کلمات (Bag of Words)

کجا کم می‌آورد؟

ترتیب را دور می‌ریزد!
«سگ، مرد را گاز گرفت» و برعکسش
دقیقا یک کیسه‌ی یکسان دارند

ایده‌ی ساده

فقط بشمار هر کلمه چند بار آمده
«فیلم عالی بود» ← فیلم: ۱، عالی: ۱، بود: ۱
برای فیلتر اسپم و دسته‌بندی ساده کافی است

تعبیه‌ی کلمات (Embedding): معنا به صورت مختصات

- هر کلمه ← یک نقطه در فضایی چندصدبعدی (مثلا ۳۰۰ عدد)
- کلمات هم‌معنا همسایه‌اند: «گره» و «سگ» نزدیک، «گره» و «پنجره» دور
- جهت‌های فضا معنا پیدا می‌کنند: جهت جنسیت، جمع‌بودن، زمان
- این مختصات را کسی دستی ننوشته، از دل متن یاد گرفته شده

جای تصویر

نمودار پراکندگی کلمات در فضای دوبعدی (خروجی t-SNE)

یک جدول یادگرفتنی

تعبیه (Embedding) یعنی: به هر کلمه‌ی واژگان، یک ردیف عدد اختصاص بده و آن ردیف را یاد بگیر.

بعد ۱ بعد ۲ بعد ۳ تا ۳۰۰

گربه	۰٫۹۱	-۰٫۲۴	۰٫۶۷	...
سگ	۰٫۸۸	-۰٫۳۱	۰٫۷۲	...
چای	-۰٫۷۶	۰٫۵۲	۰٫۱۱	...
دروازه	۰٫۰۵	۰٫۸۳	-۰٫۴۹	...

این جدول یعنی چه؟

هر سطر = یک کلمه

هر سطر ۳۰۰ عدد دارد

۱۰ هزار کلمه $\times$ ۳۰۰ =

۳ میلیون عدد یادگرفتنی

این اعداد، پارامترهای مدل هستند؛

درست مثل وزن‌های شبکه عصبی

در آغاز، این جدول کاملاً تصادفی است؛ آموزش، آن را به نقشه‌ی معنا تبدیل می‌کند.

یک تعبیه‌ی ساده: شش کلمه، دو بعد

کلمه	بعد ۱	بعد ۲	نزدیک‌ترین همسایه
گربه	۰٫۹۰	۰٫۸۰	سگ
سگ	۰٫۸۵	۰٫۹۰	گربه
اسب	۰٫۷۰	۰٫۹۵	سگ
چای	-۰٫۸۰	۰٫۶۰	قهوه
قهوه	-۰٫۸۵	۰٫۵۰	چای
آب	-۰٫۷۰	۰٫۷۰	چای

بعد اول «جاندار یا نوشیدنی» را کد کرده است: جانداران مثبت، نوشیدنی‌ها منفی.

جهت‌ها معنا دارند

۱. مختصات

$(\circ/9, \circ/2) = \text{King}$
 $(\circ/9, \circ/8) = \text{Queen}$
 $(\circ/3, \circ/2) = \text{Man}$
 $(\circ/3, \circ/8) = \text{Woman}$

۲. جهت جنسیت

$\text{Queen} - \text{King} = (\circ, \circ/6)$
 $\text{Woman} - \text{Man} = (\circ, \circ/6)$

۳. محاسبه روی بعدها

$\text{King} - \text{Man} + \text{Woman} =$
 $(\circ/6, \circ) + (\circ/3, \circ/8)$
 $= (\circ/9, \circ/8) = \text{Queen}$

جادوی حساب با کلمات

King – Man + Woman $\approx$ Queen

Paris – France + Iran $\approx$ Tehran

وقتی معنا مختصات پیدا کند، می‌توان با کلمات جمع و تفریق کرد.

این روابط را مدل خودش کشف کرده است.

تعبیه در کد: فقط یک جدول

در هر مدل زبانی، لایه‌ی اول همین است — و چیز عجیبی نیست:

```
import torch

V, d = 10000, 300          # واژگان، بُعد
emb = torch.nn.Embedding(V, d)

print(emb.weight.shape)
# torch.Size([10000, 300]) ← همان جدول

ids = torch.tensor([5, 82, 7])  # سه توکن
vecs = emb(ids)
print(vecs.shape)              # [3, 300]

# این جدول با گرادیان کاهشی آموزش می‌بیند:
print(emb.weight.requires_grad)  # True
```

چیزی جز یک ماتریس نیست Embedding

فراخوانی‌اش = برداشتن سطرها (lookup)

یعنی: این اعداد پارامترند requires_grad

پس در حین آموزشِ هر مدل، تعبیه‌ها هم بهتر می‌شوند

مدلی که هدفش فقط ساختن همین جدول است: word2vec

تفاوت word2vec با مدل‌های امروزی: آن‌جا جدول هدف بود، این‌جا فقط اولین لایه است.

word2vec دقیقا چه می‌کند؟

- یک شبکه‌ی عصبی بسیار کوچک که فقط یک تمرین انجام می‌دهد:
- «کلمه‌ای به تو می‌دهم؛ حدس بزن چه کلماتی کنارش می‌آیند»
- برای این تمرین به هیچ برچسب انسانی نیاز نیست، خود متن، تمرین و پاسخ را می‌سازد
- کسی به مدل نمی‌گوید «چای» و «قهوه» شبیه‌اند؛ اما هر دو کنار «داغ» و «نوشید» می‌آیند
- پس مدل مجبور می‌شود بردارهای مشابهی به آن‌ها بدهد تا حدسش درست دربیاید
- محصول نهایی، خود حدس‌زدن نیست؛ بردارهایی است که در مسیر ساخته می‌شوند

قدم اول word2vec: ساختن داده‌ی آموزشی

پنجره‌ای به شعاع ۲ روی جمله می‌لغزد؛ کلمه‌ی وسط «مرکزی» و بقیه «همسایه» هستند:



کلمه‌ی مرکزی: «سرد». همسایه‌ها: «هوا»، «خیلی»، «است»، «و»

جفت‌های آموزشی تولیدشده

(سرد ← هوا)

(سرد ← خیلی)

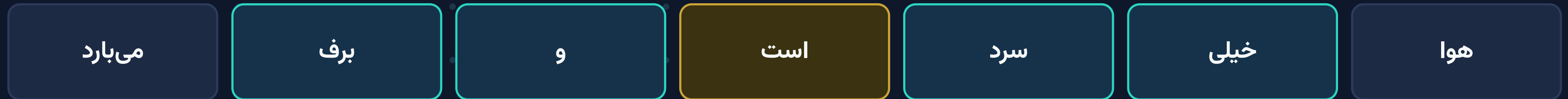
(سرد ← است)

(سرد ← و)

پنجره یک کلمه جلو می‌رود و این کار تکرار می‌شود؛ از یک متن میلیون‌کلمه‌ای، میلیون‌ها جفت به دست می‌آید.

قدم اول word2vec: ساختن داده‌ی آموزشی

پنجره‌ای به شعاع ۲ روی جمله می‌لغزد؛ کلمه‌ی وسط «مرکزی» و بقیه «همسایه» هستند:



کلمه‌ی مرکزی: «است». همسایه‌ها: «خیلی»، «سرد»، «و»، «برف»

جفت‌های آموزشی تولیدشده

- (است ← خیلی)
- (است ← سرد)
- (است ← و)
- (است ← برف)

پنجره یک کلمه جلو می‌رود و این کار تکرار می‌شود؛ از یک متن میلیون‌کلمه‌ای، میلیون‌ها جفت به دست می‌آید.

معماری word2vec: ساده‌تر از آن‌که فکر کنید

یک شبکه‌ی دو لایه، بدون هیچ تابع فعال‌سازی در میانه – واژگان فرضی: ۱۰ هزار کلمه

ورودی: One-hot

بردار ۱۰,۰۰۰ تایی
فقط یک خانه ۱ است

ماتریس تعبیه

$10,000 \times 300$
همان جدولی که می‌خواهیم!

لایه‌ی پنهان

بردار ۳۰۰ تایی
= سطر متناظر کلمه

خروجی: Softmax

احتمال ۱۰,۰۰۰ کلمه
برای همسایه‌بودن

نکته‌ی کلیدی

ضرب بردار One-hot در ماتریس تعبیه، یعنی «برداشتن یک سطر از جدول» نه محاسبه‌ی سنگین.
پس آن ماتریس $300 \times 10,000$ چیزی نیست جز فهرست بردارهای همه‌ی کلمات: خروجی واقعی ما.

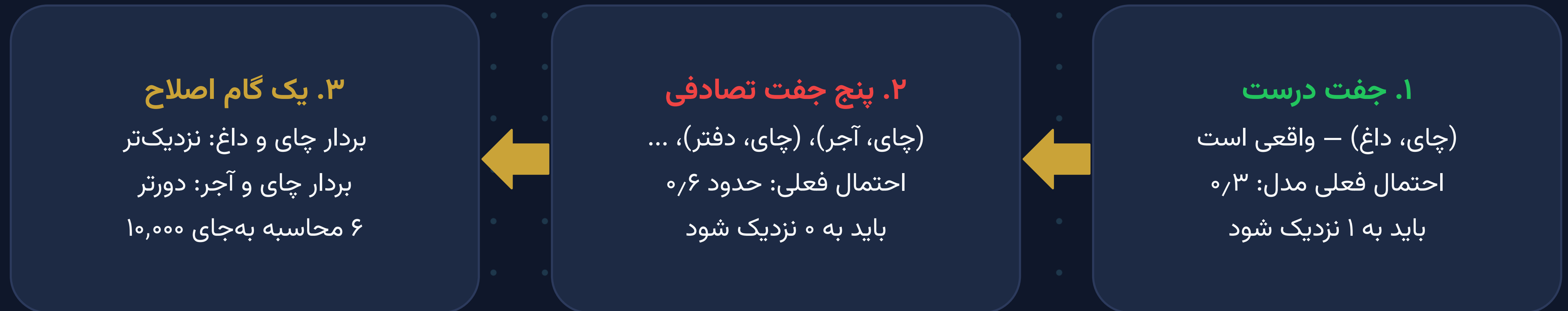
شبکه را برای حدس همسایه آموزش می‌دهیم، بعد شبکه را دور می‌ریزیم و جدول تعبیه را برمی‌داریم!

هدف دقیق word2vec، بی‌ابهام

- برای جفت (مرکزی، همسایه)، مدل باید احتمال همسایه را بالا ببرد
- این احتمال چطور حساب می‌شود؟ ضرب داخلی دو بردار، سپس Softmax روی کل واژگان
- یعنی: «شبهات چای و داغ» تقسیم بر «مجموع شبهات چای با همه‌ی ۱۰ هزار کلمه»
- هزینه‌ی این کار سرسام‌آور است: برای هر جفت، ۱۰ هزار ضرب داخلی ۳۰۰ بعدی
- با ۱۰۰ میلیون جفت آموزشی، عملاً غیرممکن می‌شود
- راه‌حل مقاله‌ی ۲۰۱۳: مسئله را عوض کن – نمونه‌گیری منفی

نمونه‌گیری منفی: میان‌برِ نجات‌بخش

به جای Softmax روی ۱۰ هزار کلمه، فقط یک سوال ساده‌ی بله/خیر برای شش جفت:



مسئله از «کدام‌یک از ده هزار کلمه؟» به «آیا این جفت واقعی است یا ساختگی؟» تبدیل شد.

کلمات منفی تصادفی نیستند: کلمات پرتکرار شانس بیشتری دارند، اما نه به اندازه‌ی فراوانی‌شان.

آموزش با اعداد: یک گام یادگیری

جفت آموزشی «(چای ← داغ)» – می‌خواهیم مدل «داغ» را بعد از «چای» محتمل بداند:

۱. وضع فعلی

بردارهای چای و داغ
تصادفی و بی‌ربط‌اند
ضرب داخلی ≈ 0



۲. تابع هزینه

می‌گویید: این ضرب داخلی
باید بزرگ‌تر شود
خطا زیاد است



۳. گرادینان کاهش

هر دو بردار کمی
به سمت هم چرخانده می‌شوند
شباهتشان زیاد می‌شود

این کار برای میلیون‌ها جفت تکرار می‌شود؛ کلماتی که هم‌نشین‌های مشترک دارند، کنار هم جمع می‌شوند.

نمونه‌گیری منفی: هم‌زمان چند کلمه‌ی تصادفی مثل «آجر» را «دور» می‌کنیم تا همه‌چیز به هم نچسبد.

دو نسخه‌ی word2vec

CBOW

از همسایه‌ها، کلمه‌ی مرکزی را حدس بزن
• هوا، خیلی، است ← «سرد»
• سریع‌تر آموزش می‌بیند

Skip-gram

از کلمه‌ی مرکزی، همسایه‌ها را حدس بزن
• «سرد» ← هوا، خیلی، است
• برای کلمات کم‌تکرار بهتر است

هر دو به یک نتیجه می‌رسند: جدولی که در آن، فاصله‌ی بردارها یعنی فاصله‌ی معنایی.

ابریپارامترهای word2vec در عمل

ابریپارامتر	مقدار رایج	اثرش چیست؟
اندازه‌ی پنجره	۵ کلمه	پنجره‌ی کوچک: شباهت نحوی — بزرگ: شباهت موضوعی
بُعد بردار	۱۰۰ تا ۳۰۰	بزرگ‌تر: دقیق‌تر ولی کندتر و داده‌خواه‌تر
حداقل تکرار	۵ بار	کلمات خیلی نادر حذف می‌شوند
تعداد نمونه‌ی منفی	۵ تا ۲۰	بیشتر: آموزش پایدارتر، کندتر
حذف پرتکرارها	«از»، «به»، «و»	کلمات بی‌محتوا کم‌تر نمونه‌گیری می‌شوند

با کتابخانه‌ی gensim، آموزش word2vec روی یک متن فارسی چند خط کد و چند دقیقه است.

تعبیه‌ی زیرکلمه‌ای – نجات فارسی: FastText

چرا برای فارسی مهم است؟

«می‌روم»، «می‌رود»، «رفتیم» یک ریشه دارند
کلمه‌ی دیده‌نشده هم بردار می‌گیرد
غلط املایی دیگر مدل را کور نمی‌کند

ایده

کلمه را به تکه‌های حرفی بشکن: «کتاب» ← کت، تا، اب
بردار کلمه = جمع بردارهای تکه‌هایش
هم‌ریشه‌ها خودبه‌خود شبیه می‌شوند

همیشه چیزی برای گفتن دارد FastText برای هر کلمه‌ی نادیده هیچ بردار ندارد؛ word2vec

دستآورد و مرزها: word2vec

کجا کم آورد؟

ایستا: «شیر» یک بردار دارد، سه معنا
کلمه‌ی نادیده = هیچ بردار
ترتیب و نحو را نمی‌بیند

چه چیزی یاد گرفت؟

هم‌معنایی: پزشک ↔ دکتر
روابط قیاسی: پاریس ↔ فرانسه مثل تهران ↔ ایران
خوشه‌های موضوعی: ورزش، پزشکی، آشپزی

این محدودیت‌ها دقیقاً همان چیزی است که در ادامه‌ی جلسه، توجه و ترنسفورمر حلش می‌کنند.

شباهت کسینوسی: با عدد بسنجیم

بردارهای دوبعدی فرضی: گربه = $(0.6, 0.8)$ ، سگ = $(0.8, 0.6)$ ، پنجره = $(-0.5, 0.9)$

۱. ضرب داخلی

$$\begin{aligned} & \text{گربه} \cdot \text{سگ} \\ &= 0.8 \times 0.6 + 0.6 \times 0.8 \\ &= 0.96 \end{aligned}$$

۲. تقسیم بر طولها

$$\begin{aligned} & \text{طول هر دو بردار} = 1 \\ & \cos(\text{گربه}, \text{سگ}) = 0.96 \\ & \text{تقریباً هم جهت!} \end{aligned}$$

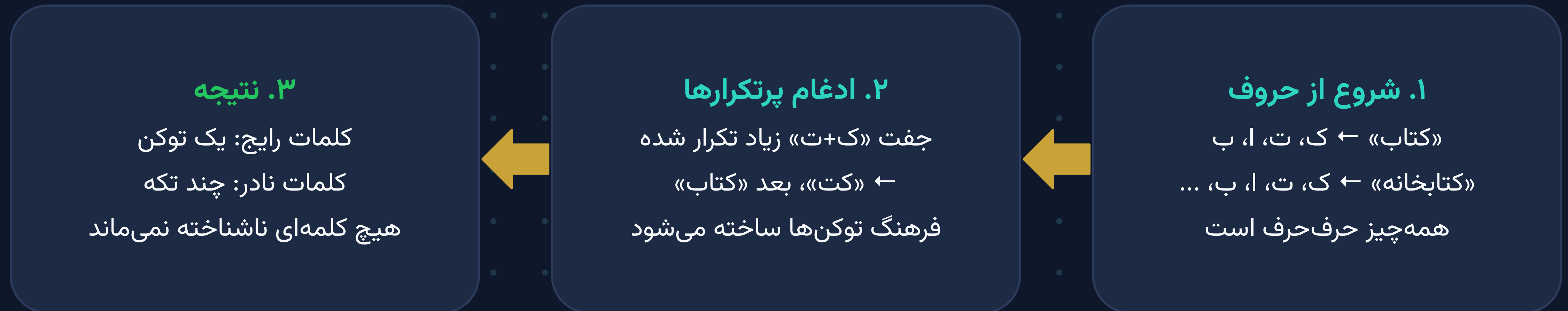
۳. مقایسه

$$\begin{aligned} & \cos(\text{گربه}, \text{پنجره}) \approx 0.14 \\ & \text{تقریباً بی ربط} \\ & \text{عدد، شهود ما را تایید کرد} \end{aligned}$$

شباهت کسینوسی یعنی زاویه‌ی بین بردارها؛ در ۳۰۰ بعد هم همین فرمول است.

توکن‌سازی: واحد کار مدل، «زیرکلمه» است

الگوریتم BPE پرتکرارترین جفت همسایه را بارها ادغام می‌کند –



مدل‌های چندزبانه یک فرهنگ مشترک دارند؛ برای همین کلمات فارسی گاهی به تکه‌های ریز می‌شکنند.

ترتیب مهم است: شبکه‌های بازگشتی (RNN)

- ایده: جمله را کلمه‌به‌کلمه بخوان و یک «حافظه» با خود ببر
- هر کلمه‌ی جدید، حافظه را به‌روز می‌کند – شبیه خواندن خودمان
- سال‌ها بهترین ابزار ترجمه و تشخیص گفتار بود
- مشکل ۱: در جمله‌های بلند، اول جمله تا آخر کمرنگ می‌شود (فراموشی)
- مشکل ۲: کلمه‌به‌کلمه یعنی گُند – موازی‌سازی ممکن نیست

ایده‌ای که همه‌چیز را عوض کرد

به جای حافظه‌ی فشرده،

به همه‌ی کلمات هم‌زمان «توجه» کن!

سازوکار توجه: پرسش، کلید، محتوا

- هر توکن سه بردار می‌سازد: پرسش Q ، کلید K ، و محتوا V
- تشبیه جست‌وجو: عبارت جست‌وجوی شما (Q) با عنوان صفحه‌ها (K) تطبیق می‌خورد و محتوا (V) برمی‌گردد
- امتیاز توجه توکن i به j = ضرب داخلی Q_i در K_j – بزرگ یعنی مرتبط
- امتیازها با Softmax به وزن تبدیل می‌شوند؛ خروجی = میانگین وزنی V ها
- هر سه بردار با ماتریس‌های یادگرفتنی ساخته می‌شوند – باز هم گرادیان کاهشی

توجه روی جمله‌ی کامل، قاب به قاب

جمله: «بازیکن توپ را به دروازه زد و آن را لرزاند» – پرسش: «آن» به چه چیزی برمی‌گردد؟

	بازیکن	توپ	را	به	دروازه	زد	و	آن	را	لرزاند
امتیاز	؟	؟	؟	؟	؟	؟	؟	؟	؟	؟
وزن	...	...	...	...	...	...	...	...	...	...

قانون بازی

برای توکن «آن» یک بردار پرسش (Q) داریم
برای هر ده توکن یک بردار کلید (K)
امتیاز هر جفت = ضرب داخلی Q در K
سپس Softmax، سپس ترکیب محتواها (V)

توجه روی جمله‌ی کامل، قاب‌به‌قاب

جمله: «بازیکن توپ را به دروازه زد و آن را لرزاند» – پرسش: «آن» به چه چیزی برمی‌گردد؟

	بازیکن	توپ	را	به	دروازه	زد	و	آن	را	لرزاند
امتیاز	۲/۵	؟	؟	؟	؟	؟	؟	؟	؟	؟
وزن	...	...	...	...	...	...	...	...	...	...

امتیاز برای «بازیکن»

ضرب داخلی پرسشِ «آن» در کلیدِ «بازیکن» ← امتیاز ۲/۵
فاعل جمله است و می‌تواند مرجع ضمیر باشد، اما...

توجه روی جمله‌ی کامل، قاب‌به‌قاب

جمله: «بازیکن توپ را به دروازه زد و آن را لرزاند» – پرسش: «آن» به چه چیزی برمی‌گردد؟

	بازیکن	توپ	را	به	دروازه	زد	و	آن	را	لرزاند
امتیاز	۲/۵	۳	؟	؟	؟	؟	؟	؟	؟	؟
وزن	...	...	...	...	...	...	...	...	...	...

امتیاز برای «توپ»

ضرب داخلی پرسش «آن» در کلید «توپ» ← امتیاز ۳/۰
رقیب جدی! توپ هم جسمی است که در جمله حرکت کرده

توجه روی جمله‌ی کامل، قاب‌به‌قاب

جمله: «بازیکن توپ را به دروازه زد و آن را لرزاند» – پرسش: «آن» به چه چیزی برمی‌گردد؟

	بازیکن	توپ	را	به	دروازه	زد	و	آن	را	لرزاند
امتیاز	۲/۵	۳	؟	؟	۴	؟	؟	؟	؟	؟
وزن	...	...	...	...	...	...	...	...	...	...

امتیاز برای «دروازه»

ضرب داخلی پرسشِ «آن» در کلیدِ «دروازه» ← امتیاز ۴/۰

بالاترین امتیاز: چیزی که می‌تواند «بلرزد»

توجه روی جمله‌ی کامل، قاب‌به‌قاب

جمله: «بازیکن توپ را به دروازه زد و آن را لرزاند» – پرسش: «آن» به چه چیزی برمی‌گردد؟

	بازیکن	توپ	را	به	دروازه	زد	و	آن	را	لرزاند
امتیاز	۲/۵	۳	؟	؟	۴	۱	؟	؟	؟	۲
وزن	...	...	...	...	...	...	...	...	...	...

فعل‌ها: «زد» و «لرزاند»

امتیاز «زد» برابر ۱/۰ و امتیاز «لرزاند» برابر ۲/۰

فعلِ خودِ عبارت مهم‌تر است: «آن را لرزاند»

توجه روی جمله‌ی کامل، قاب‌به‌قاب

جمله: «بازیکن توپ را به دروازه زد و آن را لرزاند» – پرسش: «آن» به چه چیزی برمی‌گردد؟

	بازیکن	توپ	را	به	دروازه	زد	و	آن	را	لرزاند
امتیاز	۲/۵	۳	۰/۳	۰/۳	۴	۱	۰/۳	؟	۰/۳	۲
وزن	...	...	...	...	...	...	...	...	...	...

کلمات دستوری

«را»، «به»، «و»، «را» – هر کدام حدود ۰/۳
نقش دستوری دارند، اما معنای مستقلی ندارند

توجه روی جمله‌ی کامل، قاب‌به‌قاب

جمله: «بازیکن توپ را به دروازه زد و آن را لرزاند» – پرسش: «آن» به چه چیزی برمی‌گردد؟

	بازیکن	توپ	را	به	دروازه	زد	و	آن	را	لرزاند
امتیاز	۲/۵	۳	۰/۳	۰/۳	۴	۱	۰/۳	۱/۵	۰/۳	۲
وزن	...	...	...	...	...	...	...	...	...	...

خودِ «آن»

ضرب داخلی پرسش و کلیدِ خودِ توکن ← امتیاز ۱/۵
هر توکن به خودش هم امتیاز می‌دهد

توجه روی جمله‌ی کامل، قاب‌به‌قاب

جمله: «بازیکن توپ را به دروازه زد و آن را لرزاند» – پرسش: «آن» به چه چیزی برمی‌گردد؟

	بازیکن	توپ	را	به	دروازه	زد	و	آن	را	لرزاند
امتیاز	۲/۵	۳	۰/۳	۰/۳	۴	۱	۰/۳	۱/۵	۰/۳	۲
وزن	۱۱%	۱۹%	۱%	۱%	۵۲%	۳%	۱%	۴%	۱%	۷%

روی هر ده امتیاز Softmax

دروازه ۵۱٪ – توپ ۱۹٪ – مرد ۱۱٪

لرزاند ۸٪ – آن ۴٪ – زد ۳٪

کلمات دستوری، هرکدام ۱٪

جمع دقیقاً ۱۰۰٪

توجه روی جمله‌ی کامل، قاب‌به‌قاب

جمله: «بازیکن توپ را به دروازه زد و آن را لرزاند» – پرسش: «آن» به چه چیزی برمی‌گردد؟

	بازیکن	توپ	را	به	دروازه	زد	و	آن	را	لرزاند
امتیاز	۲/۵	۳	۰/۳	۰/۳	۴	۱	۰/۳	۱/۵	۰/۳	۲
وزن	۱۱%	۱۹%	۱%	۱%	۵۲%	۳%	۱%	۴%	۱%	۷%

ترکیب محتواها (۷)

خروجی «آن» = میانگین وزنی ۷ همه‌ی توکن‌ها
$$\dots + (توپ) \times ۰,۱۹ + (دروازه) \times ۰,۵۱$$

بردار حاصل، بیش از همه به «دروازه» شبیه است
«آن» حالا معنای دروازه را حمل می‌کند

پایان!

توجه روی جمله‌ی کامل، قاب‌به‌قاب

جمله: «بازیکن توپ را به دروازه زد و آن را لرزاند» – پرسش: «آن» به چه چیزی برمی‌گردد؟

	بازیکن	توپ	را	به	دروازه	زد	و	آن	را	لرزاند
امتیاز	۲/۵	۳	۰/۳	۰/۳	۴	۱	۰/۳	۱/۵	۰/۳	۲
وزن	۱۱%	۱۹%	۱%	۱%	۵۲%	۳%	۱%	۴%	۱%	۷%

چه چیزی یاد گرفتیم؟

مدل قاعده‌ی دستوری حفظ نکرده است
از میلیون‌ها جمله آموخته که «لرزاندن»
با «دروازه» سازگارتر است تا «توپ»
و توپ ۱۹٪ باقی می‌ماند: عدم قطعیتِ صادقانه

ماتریس توجه کامل: ده در ده

هر سطر یک پرسشگر، هر ستون یک کلید – اعداد بر حسب درصد، جمع هر سطر ۱۰۰:

پرسشگر ↓	بازیکن	توپ	را	به	دروازه	زد	و	آن	را	لرزانده
بازیکن	۴۳	۱۰	۳	۳	۳	۲۶	۳	۳	۳	۳
توپ	۷	۳۳	۱۱	۳	۳	۳۱	۳	۳	۳	۳
را	۳	۵۳	۷	۳	۳	۱۹	۳	۳	۳	۳
به	۴	۴	۴	۷	۵۳	۱۲	۴	۴	۴	۴
دروازه	۳	۳	۳	۱۱	۳۳	۱۹	۳	۳	۳	۱۹
زد	۲۸	۲۷	۳	۳	۱۷	۱۰	۳	۳	۳	۳
و	۶	۶	۶	۶	۶	۲۴	۱۳	۶	۶	۲۱
آن	۱۱	۱۹	۱	۱	۵۲	۳	۱	۴	۱	۷
را	۳	۳	۳	۳	۳	۳	۳	۴۶	۶	۲۷
لرزانده	۳	۳	۳	۳	۲۵	۳	۳	۳۹	۹	۹

سطر «آن» همان چیزی است که قاب‌به‌قاب ساختیم؛ بقیه‌ی سطرها هم‌زمان و با همان یک ضرب ماتریسی حساب می‌شوند.

دقت کنید: «را»ی اول به «توپ» می‌چسبد و «را»ی دوم به «آن» – مدل نقش دستوری را از بافت فهمیده است.

ابهام لفظی: «شیر» یعنی چه؟

یک نوشتار، سه معنای کاملاً متفاوت – انسان بی‌درنگ تشخیص می‌دهد، چون بافت را می‌بیند:

شیر آب

«شیر آب را ببند»
هم‌نشین‌ها: آب، ببند

حیوان

«شیر در جنگل غرید»
هم‌نشین‌ها: جنگل، غرید

نوشیدنی

«گربه شیر را نوشید»
هم‌نشین‌ها: گربه، نوشید

ابهام استثنا نیست، قاعده است: «شانه»، «دوش»، «گل»، «سیر»... زبان پر از این‌هاست.

نقطه ضعف word2vec: یک بردار برای سه معنا

پیامد عملی

- در جست‌وجو: «شیر آب» نتیجه‌ی حیوان می‌آورد
- در ترجمه: انتخاب معادل غلط
- معنا بدون بافت، ناقص است

تعبیه‌ی ایستا (Static)

- برای «شیر» فقط یک بردار دارد word2vec
- این بردار میانگینِ همه‌ی معناهاست
- نه شیرِ جنگل است، نه شیرِ لیوان!

این دقیقا همان محدودیتی بود که ترنسفورمرها آن را برداشتند.

توجه، ابهام را حل می‌کند

در هر جمله، توکن «شیر» به کدام کلمه بیشترین توجه را می‌کند؟ (وزن‌های توجهِ سطرِ «شیر»)

شیر = نوشیدنی	نوشید	را	شیر	گربه	جمله‌ی ۱
	۵۰%	۴%	۲۵%	۲۱%	
شیر = حیوان	غرید	جنگل	در	شیر	جمله‌ی ۲
	۴۸%	۲۶%	۴%	۲۲%	
شیر = شیر آب	ببند	را	آب	شیر	جمله‌ی ۳
	۲۳%	۴%	۵۳%	۲۰%	

یک توکن، سه بافت، سه الگوی توجه کاملاً متفاوت، و در نتیجه سه بردار خروجی متفاوت.

تعبیه‌ی بافتاری: بردار هر کلمه، در هر جمله فرق می‌کند

- ورودی لایه‌ی اول برای هر سه «شیر» یکسان است – همان بردار ایستا
- اما بعد از لایه‌ی توجه، هر «شیر» ترکیبی از همسایه‌های خودش را در خود دارد
- شیرِ جمله‌ی اول به «نوشید» نزدیک می‌شود، شیرِ جمله‌ی دوم به «غرید»
- با هر لایه‌ی بعدی، این تخصصی‌شدن عمیق‌تر می‌شود
- به همین دلیل به خروجی BERT می‌گویند «تعبیه‌ی بافتاری» (Contextual Embedding)

از بردار تا Q و K و V: سه ماتریس یادگرفتنی

- هر توکن یک بردار دارد (Embedding + مکان). این بردار به تنهایی نه پرسش است نه کلید
- سه ماتریس وزن یادگرفتنی داریم: WK، WQ و WV
- بردار توکن را در هر سه ضرب می‌کنیم ← سه نقش متفاوت از یک توکن
- «پیام واقعی من»: V – «من چه چیزی عرضه می‌کنم؟»: K – «من دنبال چه می‌گردم؟»: Q
- این سه ماتریس، تنها چیزی هستند که در لایه‌ی توجه آموزش می‌بینند

چرا تقسیم بر جذر بُعد؟ (Scaled Dot-Product)

اگر امتیازها خیلی بزرگ شوند، Softmax اشباع می‌شود و گرادیان تقریباً صفر – یاد جلسه‌ی ۷!

درمان: تقسیم بر ۸

- جذر ۶۴ برابر ۸ است
- امتیازها: ۱ و ۰/۵ و ۰/۲۵
- و ۲۹٪ و ۲۳٪ – متعادل Softmax: ۴۸٪

مشکل: امتیازهای بزرگ

- در بُعد ۶۴، ضرب داخلی جمع ۶۴ حاصل ضرب است
- امتیازها بزرگ می‌شوند: مثلاً ۸ و ۴ و ۲
- و ۲٪ و ۰٪ – تقریباً قطعی Softmax: ۹۸٪

فرمول نهایی: توجه $= V \times \text{Softmax}((Q \cdot K) \div \sqrt{d})$ – آن تقسیم کوچک، آموزش را نجات می‌دهد.